fintrust.

FinTrust Project Master

Transaction Monitoring & Risk Alert Platform

RELEASE 1 | 13 October 2026

Source checkpoint 48399b9

Authoritative consolidated documentation. Explicitly separates local verification, historical deployment evidence and pending live gates.

Fictional client: Imbokodo Bank. No real bank adoption or production compliance certification is claimed.

Contents

1. Executive Summary 4

2. Imbokodo Bank Case Study 4

3. Problem Statement 4

4. Product North Star 4

5. Project Scope 5

6. Assumptions and Constraints 5

7. Stakeholders 5

8. Users and Role Separation 5

9. Functional Requirements 6

10. Non-functional Requirements 6

11. Planning and Development Approach 6

12. End-to-End Business Flow 7

13. Transaction Lifecycle 9

14. Risk Monitoring Model 9

15. Risk Scenarios and Rules 9

16. Quarantine and Recovery 10

17. Risk Events 11

18. Alerts 11

19. Analyst Review Workflow 12

20. Accounts and Transaction History 12

21. Insights and Reporting 13

22. Audit and Traceability 13

23. Technical Failure / Retry / DLQ 13

24. AWS Architecture 14

25. Architecture Decisions and Trade-offs 16

26. S3 Storage Model 16

27. PostgreSQL Design 16

28. DynamoDB Role 18

29. Security / IAM / RBAC 18

30. POPIA / Data Governance 18

31. Retention Strategy 19

32. Cost Governance 19

33. AWS Budgets 19

34. Testing Strategy 20

35. CI/CD 21

36. Release Verification 21

37. Challenges and Failures 21

38. Lessons Learned 22

39. Current Deployment State 22

40. Future Roadmap 22

41. AI Investigation Assistant 23

42. References 23

1. Executive Summary

LOCAL VERIFIED

FinTrust is a transaction monitoring and risk investigation platform. It accepts CSV files, preserves transaction identity and source evidence, scores risk and records each review decision. It does not move funds, reverse charges or establish fraud.

Current source checkpoint 48399b9: local E2E verified through 20 scenarios and 125 selected tests, real isolated PostgreSQL, authenticated UI/API checks and mocked AWS services. The historical workbench regression remains 811 passed / 1 expected skip. Existing monthly and zero-spend budget configuration is live verified and preserved; exact-source Linux verification and live AWS E2E remain pending.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; docs/architecture/TRANSACTION_LIFECYCLE.md

2. Imbokodo Bank Case Study

PLANNED

Imbokodo Bank is a fictional client created for this project case study. Its operations team needs to reconcile transaction exports and investigate exceptions without losing the source evidence or confusing invalid data with financial risk.

The case study provides business context, not a claim of a real bank engagement, production adoption or regulatory approval.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md

3. Problem Statement

LOCAL VERIFIED

Manual file inspection can overlook invalid rows, repeated source transactions and unexplained risk decisions. Retrying partial processing can duplicate financial records or notifications unless identity and recovery are deliberate.

FinTrust separates data-quality decisions, ledger acceptance, risk assessment, investigation and technical recovery. Every submitted row needs a recorded outcome; technical failure must not silently erase accepted financial activity.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md

4. Product North Star

LOCAL VERIFIED

What happened? Why is it risky? What should I do next?

FinTrust helps a Risk Analyst answer these questions quickly, consistently and with evidence. Start with the Analyst workbench, investigate risk events and complete assigned reviews. Use Insights and Reports for supporting analysis.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; frontend/release1.html

5. Project Scope

LOCAL VERIFIED

Release 1 covers CSV admission, exact duplicate protection, positive-amount transaction validation, quarantine correction, deterministic scoring, durable review and scoped read/report/audit workflows.

Refund/reversal accounting, foreign-exchange conversion, live account balances, enterprise federation, durable notification inboxes and AI investigation are outside the implemented first release. Local direct processing does not establish cloud transport completion.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; docs/RELEASE1_OPERATIONS.md

6. Assumptions and Constraints

LOCAL VERIFIED

Use synthetic data for practice. Region remains af-south-1. The database is private; no NAT Gateway or public RDS access is part of the approved architecture. Lambda targets Python 3.12 / Linux x86_64.

Amount thresholds operate per transaction currency, with no implied FX normalization. Late arrivals do not retroactively rescore previous transactions. Historical package verification is tied to its exact source and checksum.

Source: docs/RELEASE1_OPERATIONS.md; backend/fintrust/rules.py

7. Stakeholders

PLANNED

The fictional bank operations sponsor owns business scope and approves risk policy. Analysts investigate transactions; Auditors examine evidence; Administrators govern access; Maintainers handle technical recovery.

A data owner must approve retention and disposal. A deployment owner controls account mutations and cost acceptance. These accountability assignments are the intended operating model, not evidence that real organizational appointments exist.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; docs/SECURITY.md

8. Users and Role Separation

LOCAL VERIFIED

Named local users have a stable user ID and immutable username/subject, display name, optional email, role, ACTIVE/DISABLED status, creation time and actual last sign-in time. Legacy user IDs derive deterministically from the existing subject. Unknown historical creation times remain unknown.

Admin creates named records and changes role/status with a mandatory reason and governance audit. Creation does not issue a credential. Private local enrollment uses a hidden prompt, stores only a SHA256 hash, cannot create an identity or change its role/account scope, and cannot silently replace an existing credential. The registry is operator-controlled and ignored by Git; production federation/password authentication remains a separate gate.

Backend checks deny unsupported actions on each request. Analyst investigates, corrects assigned-account quarantine and claims/completes owned reviews. Admin governs users and review assignment, never clears/confirms risk or edits financial evidence. Auditor is read-only with audit access; Maintainer handles technical recovery only. Disable blocks subsequent authentication but preserves identity, ownership and audit. Admin must explicitly reassign unresolved work; no automatic role inheritance.

Manage users in a compact table: name, username, role, status, last sign-in and credential readiness. Search filters only the already-authorized registry list; sorting never widens authority. No Datadog assets or proprietary branding are used.

Role	Responsibility
Analyst	Upload, correct quarantine, investigate and decide reviews within scope
Auditor	Read-only scoped evidence, reports and audit
Administrator	User/account governance, oversight and unknown-account cases
Maintainer	Technical diagnosis/recovery; no financial review decisions

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; backend/fintrust/auth.py

9. Functional Requirements

LOCAL VERIFIED

Record source system/reference, canonical transaction identity, fingerprint, file/batch and row provenance. Reject invalid files; quarantine invalid rows; retain all accepted risk levels.

Expose transaction/risk/review/delivery states, correction lineage and actor-attributed history. Reconcile accepted, quarantined, duplicate, pending and failed outcomes. Prevent blind duplicate notification retries.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; docs/architecture/TRANSACTION_LIFECYCLE.md

10. Non-functional Requirements

PENDING VERIFICATION

Private networking, verified TLS, least privilege and account-scoped access are required. Atomic financial commits, immutable evidence, bounded reads and deterministic scoring protect correctness.

Throughput, availability SLA, load capacity and production incident response are not established by functional test counts. Production performance has not been verified.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; docs/SECURITY.md

11. Planning and Development Approach

LOCAL VERIFIED

I verify each phase locally before requesting approval for live AWS changes. Domain rules and artifacts are frozen before deployment; read-only preflight checks assumptions before writes.

The completed local sequence covers repository structure, flow audit, lifecycle, quarantine recovery, review ownership, investigation, Analyst workbench and 20-scenario verification. Existing AWS budget

configuration is verified and preserved. Next: exact-source Linux packaging and controlled cloud integration.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; docs/architecture/REPOSITORY_STRUCTURE.md

12. End-to-End Business Flow

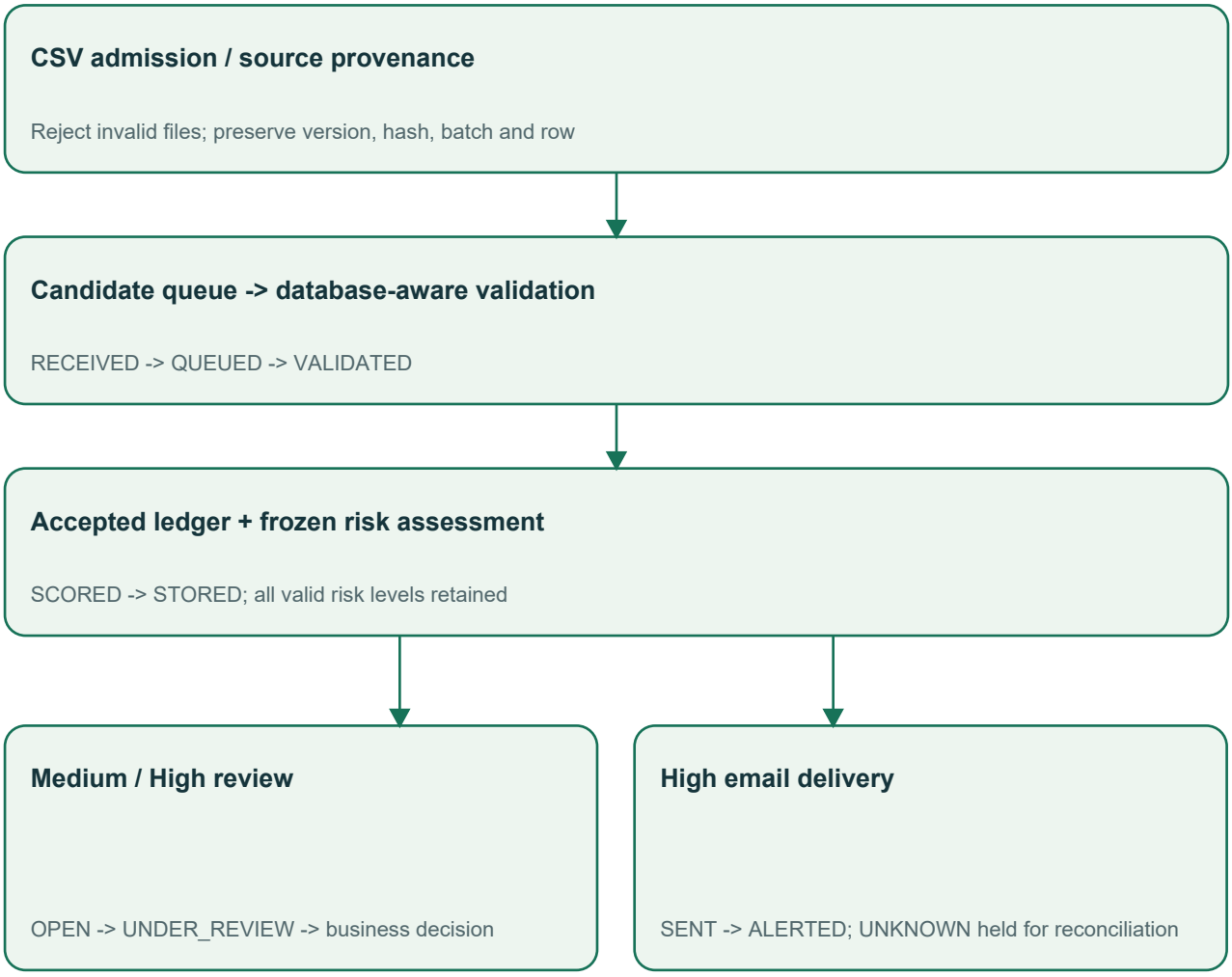
LOCAL VERIFIED

Cloud target: Analyst CSV upload -> input S3 -> ingest parsing/admission -> SQS FIFO candidates -> private consumer database-aware validation -> quarantine OR accepted ledger and frozen scoring -> DynamoDB projection. Medium/High review work is committed independently of High-only email delivery.

The local demonstration uses mocked S3 and a direct SQL consumer helper. It does not prove live FIFO, Streams or SES. Keep the original process diagrams as historical references. Use the processing order above and lifecycle contract for the current implementation.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md

Processing, review and delivery



Invalid rows branch to quarantine. Corrections create new linked occurrences and repeat validation/deduplication. Review does not wait for email.

Supporting detailed diagram: docs/presentation/diagrams/FinTrust_Activity_Diagram.pdf

13. Transaction Lifecycle

LOCAL VERIFIED

Actual processing order: RECEIVED -> QUEUED -> VALIDATED -> SCORED -> STORED. Ingest is outside the VPC; account-aware validation happens in the private consumer after candidate handoff.

Migration 003 adds transaction_lifecycle_events as append-only observations, not a competing mutable status model. Meaningful events record sequence, time and system actor. Human correction/review events retain their authenticated actor. Old rows are not backfilled with invented history.

Risk, review, delivery and processing are separate dimensions. Reads expose provenance and a bounded timeline; timeline_complete=false discloses missing or truncated history. The runtime must not be deployed before migration 003 and its service grants are approved.

Branch	Authority / meaning
Quarantine	Original occurrence terminal; correction creates linked new occurrence
Review	OPEN -> UNDER_REVIEW -> CLEARED / CONFIRMED_RISK / ESCALATED
Recovery	TECHNICAL_FAILURE -> RETRY -> RECOVERED; DLQ needs observed evidence
Projection failure	Financial state remains STORED; pending external projection is recoverable

Source: docs/architecture/TRANSACTION_LIFECYCLE.md; sql/migrations/003_transaction_lifecycle.sql

14. Risk Monitoring Model

LOCAL VERIFIED

Scoring is deterministic and records rule version, score and triggered explanations with the financial outcome. Scores are capped at 100. Low is below 40, Medium is 40 through 69, and High is 70 or above.

HIGH is valid accepted data requiring attention, not quarantine. Every accepted scored transaction has a risk_events record; only Medium/High open routine reviews. The rules support this case study; they are not a validated bank fraud model. Risk outcomes do not authorize money movement.

New frozen results record each triggered rule name, points, explanation, measured value, threshold and context. Signal creation time is the authoritative processed_transactions.processed_at timestamp. Keep historical results unchanged; do not reconstruct missing measurements. Frozen scoring semantics and rule version remain unchanged.

Source: backend/fintrust/rules.py; docs/architecture/FINTRUST_FLOW_AUDIT.md

15. Risk Scenarios and Rules

LOCAL VERIFIED

Frozen Release 1 velocity is more than 5 accepted transactions in the closed 300-second event-time window, including the current transaction. Test 5 versus 6 and the exact window boundary. The 10/11/12-per-hour rule is exploratory and deferred.

For ZAR examples, 49,999.99 and 50,000.00 do not trigger HIGH_AMOUNT; 50,000.01 does. Thresholds apply numerically per currency. Distinct similar charges stay in the ledger and can add a possible-duplicate signal; exact repeats are deduplicated.

Signal	Default points / boundary
HIGH_AMOUNT	30; amount > 50,000
VERY_HIGH_AMOUNT	25 additional; amount > 200,000
TRANSFER	20; transfer type
CROSS_BORDER	15; country differs from ZA / explicit cross-border
MERCHANT_MARKER	20; synthetic highrisk marker
HIGH_VELOCITY	30; accepted count > 5 within 300 seconds
POSSIBLE_DUPLICATE_CHARGE	40; distinct references match facts within 120 seconds

Source: backend/fintrust/rules.py; docs/RELEASE1_E2E_MANIFEST.md

16. Quarantine and Recovery

LOCAL VERIFIED

Original restricted safe evidence, reason code/detail, source version, batch and row remain immutable. Corrections are independently versioned, attributed to the named Analyst and linked to a new occurrence. The existing validator, identity checks and normal processing run again; correction never approves a ledger write directly.

Backend recovery detail exposes OPEN, CORRECTED, REVALIDATION_FAILED, ACCEPTED or REJECTED derived from existing workflow and durable outcome records. No competing status table was added. Failed attempts retain latest reason and every prior correction. Accepted and duplicate resolutions link to the canonical transaction; duplicates do not add a ledger entry.

Review original and corrected values, the correcting Analyst, correction time, attempt count, outcome and audit timeline. Evidence reads are version-pinned, bounded and account-scoped; incomplete history is explicitly marked. Missing/unknown-account cases remain Admin-visible for access resolution; Admin cannot perform the correction, and Analysts are not given blanket access to unassigned financial data.

Local correction/projection outbox retry remains durable in PostgreSQL. Mock S3 evidence is session-limited, as before; this is not a live retention proof. No credential or sensitive raw payload is logged.

Quarantine now opens a list-preserving recovery drawer with original and corrected values, issue, attempts/outcome, immutable correction history and recorded timeline. Existing account-scoped correction/revalidation actions are reused. Missing/unknown-account cases remain fail-closed under the established access model.

The compact quarantine list shows reference, reason, recovery/workflow status, account, recorded age and source provenance. Age derives from quarantined_at. The correcting Analyst is available in detail, not in the current list API. Correct / Revalidate opens the existing correction form; submission still requires normal validation, duplicate checks and processing.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; backend/fintrust/release_workflow.py; backend/fintrust/workflow_api.py

17. Risk Events

LOCAL VERIFIED

risk_events accompanies every accepted scored transaction, including Low. A risk event is evidence of assessment; it is not synonymous with a High alert or confirmed fraud.

A transaction links its frozen score and rule explanations, risk event, review state, delivery state and source occurrences. Similar charges with distinct identity remain separate ledger records for investigation.

GET /v1/risk-events/{event} assembles account-scoped transaction/source, frozen signals, owner/review, independent alert state, related history and actual audit timeline. Event data stays authoritative in PostgreSQL; delivery claims stay independent in DynamoDB. No competing risk-event state table is introduced.

Investigation drawer retains the queue and follows WHAT HAPPENED / WHEN / WHERE / WHY IS IT RISKY / NEXT STEPS. The displayed last update is the latest review-history change. Compact severity/scenario/assignee/account/status controls, server-side sorting and filtered total/HIGH/active counts operate before pagination. Related history is limited to 20 rows and explicitly reports truncation.

Account baseline is current-ledger, same-currency accepted transactions in the preceding 30 event-time days, strictly before the investigated transaction. It is labelled as contextual data, not the original scoring snapshot; there is no FX aggregation or fabricated comparison. Prior risk/review counts and related transactions come from scoped SQL.

The Analyst Overview has six scoped metrics and an active-review table sorted by score. Investigate opens the existing evidence drawer. Transaction activity, risk distribution, scenario counts and review outcomes reuse current read-only APIs. Scenario matches can overlap. High-risk alerts count flags, not email delivery. Historical unscored rows are excluded from the processed metric but remain visible in transaction activity.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; sql/schema.sql

18. Alerts

LOCAL VERIFIED

High risk enters the email path. Conditional DynamoDB claims coordinate delivery and durable SQL receipts support visibility. SENT evidence permits ALERTED; PENDING, SENDING or UNKNOWN must not be displayed as sent.

A provider timeout may mean the email was sent without a recorded response. UNKNOWN is held for reconciliation, not blindly resent. Exactly-once email is not guaranteed. Medium/High review work remains visible even if delivery fails.

Investigation displays the existing logical HIGH intent and independent PENDING / SENT / FAILED / RETRYING delivery evidence. SENDING and UNKNOWN remain conservative reconciliation states. Missing delivery evidence is not proof of sending; delivery failure does not rollback the ledger or review. SQL legacy receipts and DynamoDB delivery claims remain explicitly distinguished.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; backend/fintrust/alert_delivery.py

19. Analyst Review Workflow

LOCAL VERIFIED

Each Medium/High review is initially unassigned, with at most one named Analyst owner. Analysts may self-claim an OPEN review; a second claim or stale version is rejected. Only the current owner can make a business decision. Admin may assign, reassign or release an active review with a mandatory reason, never make its decision.

Migration 004 extends the existing transition guard for reassignment/release and adds a reviewer/status/created-time queue index. It reuses reviewer_subject, version and immutable risk_review_history. Escalated reassignment preserves the previous decision, actor, time and resolution; escalation cannot be unassigned. The new owner may resume investigation through the existing transition workflow.

PostgreSQL SELECT FOR UPDATE serializes competing mutations; expected_version rejects stale requests and request IDs support safe replay. A single-host cross-process registry lock and fresh active-role checks prevent a concurrent disable/role change from slipping into a new assignment or human mutation.

My Reviews, Unassigned, All Open, High Risk and Over SLA filters run on the scoped backend before pagination. Age is calculated from creation time. FINTRUST_REVIEW_SLA_SECONDS is optional (60 through 31536000); absent configuration yields an explicit unconfigured result, not an invented business deadline.

POST /v1/reviews/{event}/notes appends an investigation note only for the currently assigned active Analyst in UNDER_REVIEW. Expected version, row lock, request digest and replay protection reuse risk_review_history. The current note projection can change, but each prior note remains immutable in history.

Local additive migration 005_investigation_notes.sql evolves only the existing transition guard to allow a same-owner UNDER_REVIEW note append. It adds no table/column/state. Base -> 001 -> 002 -> 003 -> 004 -> 005 is tested locally; frozen 001-004 files remain unchanged. Current live runner approval does not implicitly include 003-005.

Clear / Confirm Risk / Escalate require notes, actor and resolution evidence. Wrong Analyst, Admin, Auditor and Maintainer cannot decide or append notes. Administration can assign but cannot substitute for Analyst judgment. Existing terminal/escalated recovery and ownership rules remain authoritative.

Reviews over SLA reuse the configured backend deadline. With no configured deadline, the metric shows Not configured. The Risk Events Over SLA filter uses the existing active-review endpoint; it does not create a new rule or include closed reviews. Other filters remain scoped and run before pagination.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; backend/fintrust/workflow_api.py

20. Accounts and Transaction History

LOCAL VERIFIED

Transactions and account history use scoped authorized reads, time and currency context, and stable transaction identity. Stored historical rows without assessment remain Not assessed rather than receiving invented scores.

Customer/account foreign keys protect reference integrity. Unknown accounts are quarantined, not silently created. The balance column is not a real-time banking balance engine; transaction totals are not available funds.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; sql/schema.sql

21. Insights and Reporting

LOCAL VERIFIED

Insights expose scoped SQL aggregates and exact tabular equivalents. Transaction/currency context must remain visible; cross-border activity alone is not suspicious.

Reports UI exports the current authorized query page. Backend scheduled monthly JSON/CSV outputs to S3 exist but are not a complete generated-report catalogue in the UI. A monthly summary is not a full ledger archive.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; backend/fintrust/report.py

22. Audit and Traceability

LOCAL VERIFIED

Trace from source system/reference and fingerprint through file SHA256, batch, row, canonical tx_id, frozen assessment and human decision. Ingestion outcomes and reconciliation disclose duplicates and pending failures.

Append-only lifecycle and review histories prevent silent overwrites. Local governance records are not CloudTrail or enterprise identity proof. Evidence is sanitized: no tokens, credentials or secret values belong in logs or screenshots.

User governance preserves named actor, reason and before/after metadata in existing versioned receipts. Review history exposes claim, reassignment, release and decision actions. Correction history records human submission and authoritative system revalidation/ledger outcomes. Disable and display-name changes do not rewrite past actors or delete audit.

Source: docs/architecture/TRANSACTION_LIFECYCLE.md; docs/SECURITY.md

23. Technical Failure / Retry / DLQ

LOCAL VERIFIED

A failed projection after financial commit leaves the SQL ledger STORED. Replay uses immutable identity and the frozen result, then marks RECOVERED only after durable completion. RETRY_REQUESTED by an operator is different from an executed worker retry.

FIFO batch processing stops at the first failure and reports failed plus unprocessed IDs. The transaction DLQ and async failure queues hold technical failures; they are not quarantine. Automatic lifecycle DLQ transport/reconciliation remains deferred.

A SQL outage before occurrence registration cannot create a SQL audit event. Worker transport evidence remains authority; the API never fabricates missing observations.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; docs/architecture/TRANSACTION_LIFECYCLE.md; docs/OPERATIONS.md

24. AWS Architecture

PENDING VERIFICATION

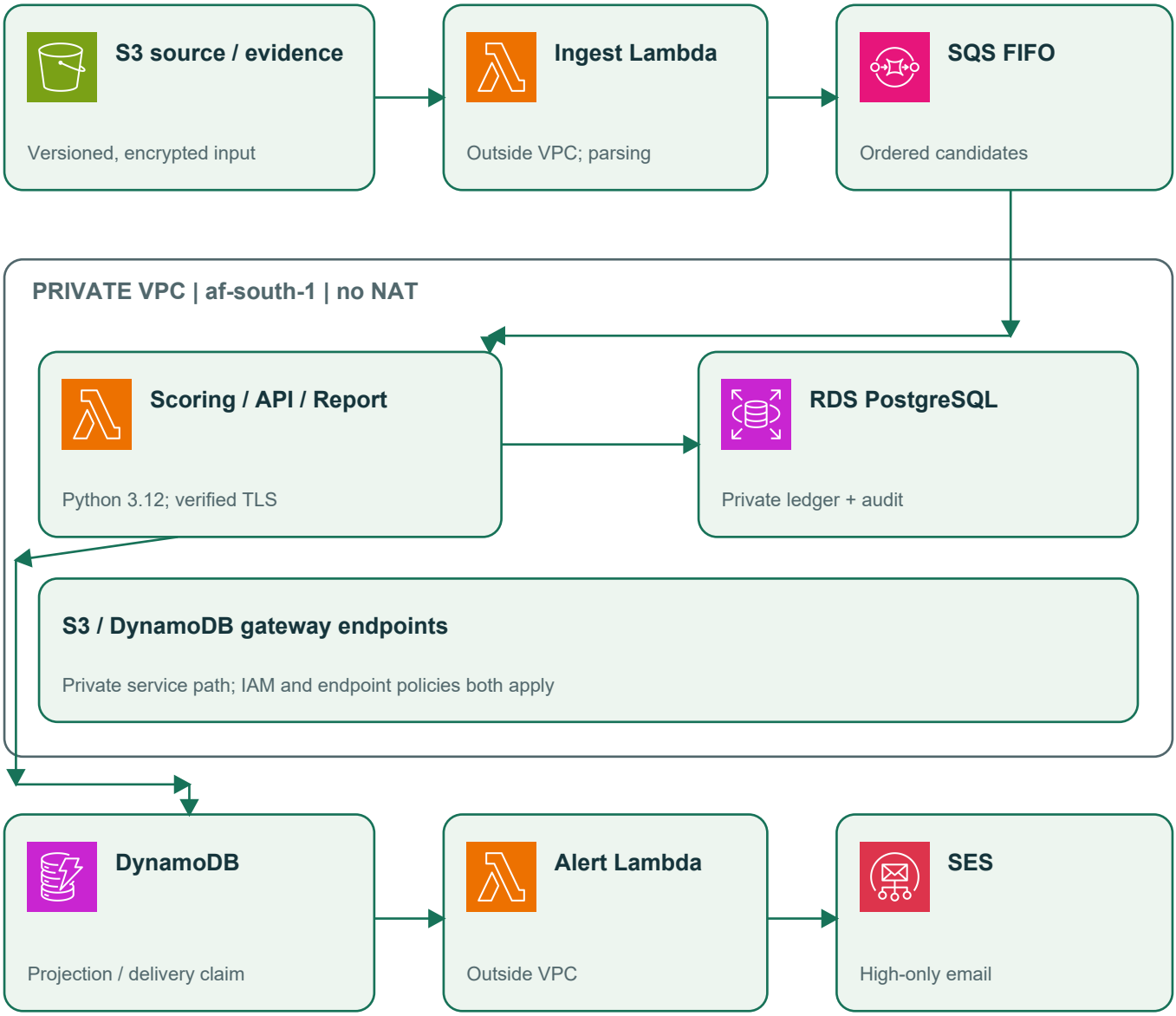
af-south-1 is preserved. Private application subnets reach the database security group on TCP 5432 only from the approved application security group. S3 and DynamoDB gateway endpoints support the private workers without NAT.

Ingest and alert run outside the VPC because their selected cloud handlers do not require SQL. Scoring, API and report are private SQL clients. SES and trigger wiring need separate live verification.

Official AWS icons in the retained diagrams identify services; they do not prove that current-release wiring is deployed. Diagram colors reflect their historical presentation checkpoint. Temporary Secrets Manager migration networking was deleted after read-only preflight.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; docs/diagrams/README.md

Architecture - official AWS icons



Official AWS icons retained in docs/diagrams/aws-icons/. Foundation historically verified; current-release wiring pending. Temporary migration interface endpoint is not standing infrastructure.

Supporting detailed diagram: docs/presentation/diagrams/FinTrust_AWS_Architecture.pdf

25. Architecture Decisions and Trade-offs

LOCAL VERIFIED

I used SQS FIFO for per-account ordering and SQL uniqueness to prevent replay beyond the FIFO deduplication window. PostgreSQL stores the authoritative ledger; DynamoDB holds immutable projections and delivery claims.

I kept NAT out of the design to avoid standing cost and limit outbound access. Private bootstrap credentials need a reviewed Secrets Manager network path. A temporary interface endpoint supported the historical read-only preflight and was cleaned up afterward.

I chose Single-AZ RDS, disabled consumers and small capacity to control development cost. Adoption, scale and HA need independent production review.

The original FinTrust brief defines the scope. External references validate implementation decisions. No AML expansion, funds authorization, account freezing or autonomous fraud/regulatory decisions were added. Industry suggestions outside the brief remain future scope.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; infrastructure/PRIVATE_RDS_MIGRATION_PLAN.md

26. S3 Storage Model

LOCAL VERIFIED

Raw input, quarantine and report buckets separate operational data classes. Stage 2 uses private, versioned buckets with SSE-KMS, secure-transport policy and a shared regional customer-managed key. Object version/checksum pinning ties package evidence to the exact uploaded artifact.

Quarantine safe payloads remain sensitive. Retained bucket/key resources and noncurrent versions can continue costing money after stack deletion. Current reports do not replace archival exports.

Source: infrastructure/storage.py; docs/OPERATIONS.md

27. PostgreSQL Design

LOCAL VERIFIED

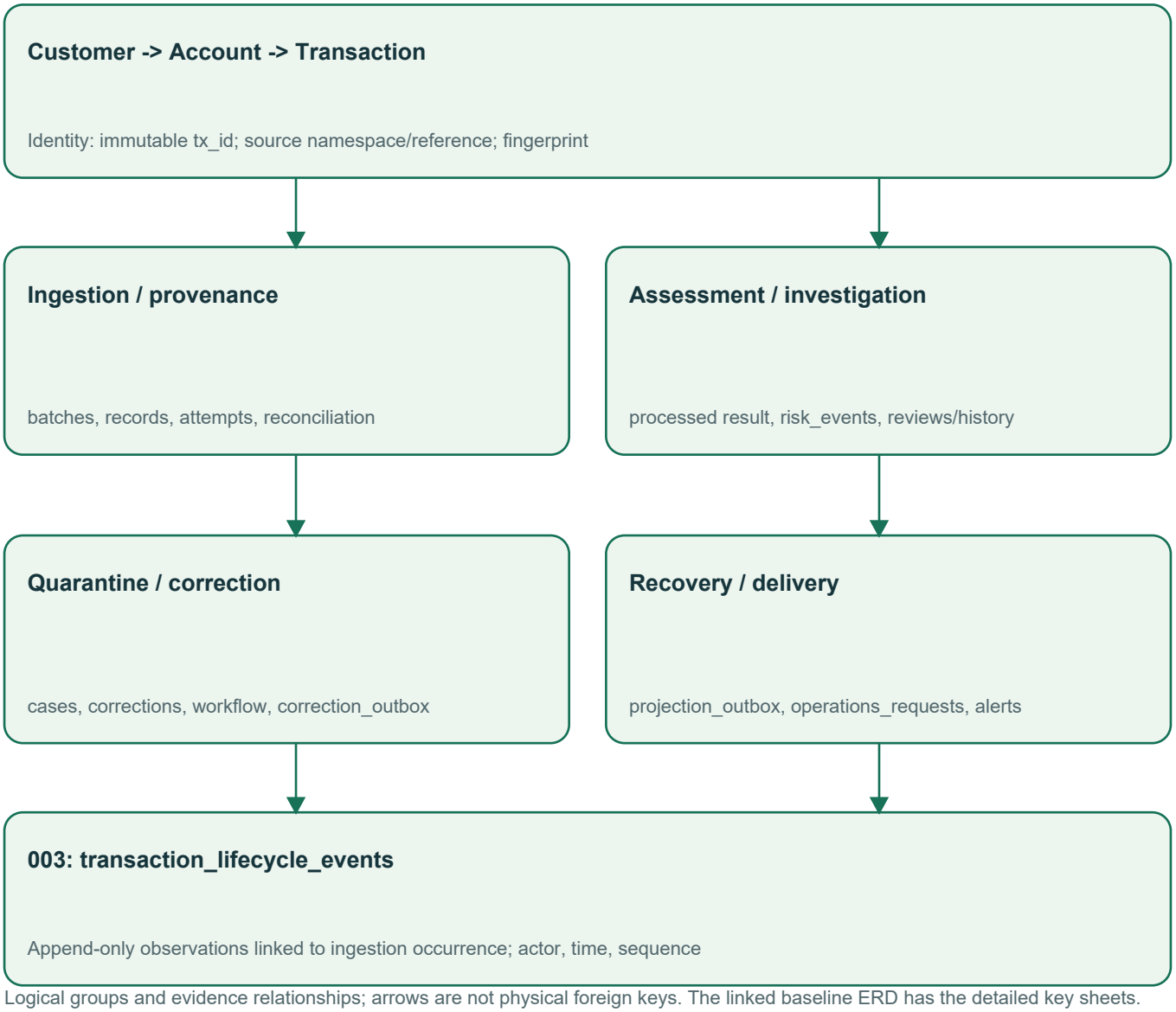
Authoritative base: the checksum-bound section of sql/schema.sql before Additive identity hardening, rendered by tools/render_release1_base.py with sql/bootstrap/release1-base-manifest.json. Six base tables include Phase 1 alert upgrades. Apply verified identity 001, workflows 002 and lifecycle 003 in order. Catalog prerequisites reject drift. Synthetic seeds are not required for live bootstrap.

Core domains: customers/accounts; transactions and frozen processed_transactions; ingestion batches/records/attempts/reconciliation; quarantine/corrections/outbox; risk events/reviews/history; alerts; projection and operations recovery.

The 17-table baseline ERD predates 003, which adds transaction_lifecycle_events. Migration 004 changes only review guard/index; no new tables. Its checksums preserve earlier history and validate the current catalog. Latest live preflight found no base schema. Live bootstrap, 001/002/003/004/005 and grants remain unapproved.

Source: sql/schema.sql; sql/migrations/003_transaction_lifecycle.sql; infrastructure/RELEASE1_DATABASE_BOOTSTRAP_REVIEW.md; docs/architecture/TRANSACTION_LIFECYCLE.md; sql/migrations/004_review_assignment.sql

Current data domains and lifecycle extension



Logical groups and evidence relationships; arrows are not physical foreign keys. The linked baseline ERD has the detailed key sheets.

Supporting detailed diagram: docs/presentation/diagrams/FinTrust_ERD.pdf

28. DynamoDB Role

LOCAL VERIFIED

DynamoDB holds immutable transaction projections for stream-driven delivery and conditional alert claims. It does not replace the PostgreSQL financial ledger or analyst review authority.

SQL projection_outbox coordinates retries when the secondary write fails after commit. The current foundation uses on-demand billing, AWS-owned encryption and no automatic TTL for this delivery state. Mocked tests do not prove live stream processing.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; infrastructure/messaging.py

29. Security / IAM / RBAC

LOCAL VERIFIED

Separate execution roles, scoped actions/resources, account authorization and service-specific database grants limit exposure. IAM rds-db:connect grants connection authority, not SQL table privileges. Database clients require TLS verify-full with the bundled RDS CA.

RDS storage uses AWS-managed aws/rds encryption; the Stage 2 S3 key is not automatically the RDS key. Managed master credentials are for reviewed bootstrap, not application environment variables or browser payloads.

Future 003 rollout must give scoring INSERT/SELECT plus sequence USAGE, API SELECT only, and no UPDATE/DELETE on lifecycle audit. The fixed live runner still supports only its reviewed phases. No broad grants should mask missing permissions.

The earlier multi-user phase changed local backend RBAC. The workbench phase changes presentation only. No AWS policy, credential, resource, deployment or activation changed. Controlled local bearer enrollment is a verification mechanism, not enterprise identity management. Existing scopes and fail-closed financial access remain authoritative.

Source: docs/architecture/TRANSACTION_LIFECYCLE.md; docs/SECURITY.md; infrastructure/iam/DEPLOYER_DESIGN.md

30. POPIA / Data Governance

PLANNED

Names and identity numbers are direct identifiers; accounts and transaction details are linkable personal financial information. Synthetic examples, restricted reads, encryption and sanitized evidence reduce exposure.

This document records design controls, not a legal opinion or certification of POPIA compliance. A data owner must approve purposes, access, retention, disposal and incident procedures before real personal data is processed. No legal retention period is asserted.

Source: docs/data_governance.md; docs/SECURITY.md

31. Retention Strategy

PLANNED

Current processing logs use 14-day retention; queues have finite retention and need investigation before expiration. The current DynamoDB delivery state has no TTL. Retained S3 versions/quarantine and RDS backups/snapshots require explicit disposal authorization.

The design proposes 12 months in the ledger, closed-period full exports in S3 Standard for 3 years and Glacier Flexible Retrieval for 7 years. This is not a legal determination and the export/purge job is not implemented. The archive/ transition after 1,095 days does not apply to monthly/ report summaries.

Older governance files describe different assumptions. Current templates and these stated limits take precedence; never delete ledger rows without reconciliation, checksum/count proof, restore test and approved policy.

Source: docs/OPERATIONS.md; infrastructure/storage.py

32. Cost Governance

LIVE INVENTORY VERIFIED

The project target is USD 10 per month. This is a budget alert target, not a spending cap or proof that the architecture fits within USD 10 before credits. Do not infer free operation from the reported budget spend.

Read-only inventory on 12-13 October 2026 confirms private Single-AZ PostgreSQL 16.15 on db.t4g.micro with 20 GiB gp3 storage, deletion protection and one-day backup retention. RDS compute/storage and retained backups are the main recurring exposure. The active RDS-managed master secret adds Secrets Manager storage/request charges; no secret value was read.

The shared customer-managed S3 KMS key, retained S3 versions/artifacts and log storage also carry recurring or request costs. SQS, DynamoDB, Lambda, SES and transfer add usage-dependent charges. Existing ingest reserved concurrency remains 0; activation is separately gated.

The project VPC has no NAT Gateway or interface endpoint. Only S3/DynamoDB gateway endpoints remain. Historical temporary preflight cleanup is preserved. No new endpoint, proxy, compute resource or billing service was created.

Prices in old stage plans remain dated estimates. Current service rates, remaining credits and a full-month pre-credit bill were not re-quoted in this configuration check. Check them before spending approval; credit-funded usage is not a permanent Free Tier entitlement.

Source: infrastructure/DEPLOYMENT_BOOTSTRAP.md; docs/OPERATIONS.md; docs/EVIDENCE/05-rds/release1-temporary-preflight-cleanup-result.md; docs/EVIDENCE/01-budget/2026-10-12-budget-readonly-inventory.json; docs/EVIDENCE/01-budget/2026-10-13-budget-control-verification.json

33. AWS Budgets

LIVE CONFIGURATION VERIFIED

Preserved the existing FinTrust-Monthly-Budget: COST, USD 10, MONTHLY, healthy, with 50%, 80% and 100% ACTUAL notifications and 100% FORECASTED. All use GREATER_THAN; the actual dollar thresholds are greater than USD 5, 8 and 10. No replacement budget was created.

Preserved My Zero-Spend Budget: MONTHLY with a USD 1 configured limit, but its alert is ACTUAL, ABSOLUTE_VALUE, GREATER_THAN USD 0.01. The alert does not wait for USD 1. Its existing cost types include credits and refunds; a zero reported spend does not mean RDS or KMS stopped charging.

Each of the five alerts has one configured email subscriber, and all use the same existing recipient. Evidence stores only the recipient fingerprint. Subscriber configuration and budget health are verified; recipient activation was not exposed by the contact API and actual email delivery was not tested. Check recipient status or receipt in the console/inbox without generating paid usage.

Both controls cover account 966467878406 across all regions. Application infrastructure remains in af-south-1; the Budgets API is an account-wide service accessed through us-east-1. Project and Environment cost-allocation tags are inactive, so no FinTrust-specific cost filter is claimed. Activation, tag normalization and complete resource attribution remain future reviewed work.

Budgets updates up to three times daily, typically 8-12 hours apart, with billing/notification delays. It warns rather than stopping resources. Read alerts promptly, inspect cost drivers and obtain approval before resource shutdown or cleanup. Monitoring-only budgets/alerts are free; no paid budget reports or automated actions were enabled.

Control	Verified configuration
FinTrust-Monthly-Budget	USD 10/month; 50/80/100% ACTUAL; 100% FORECASTED
My Zero-Spend Budget	ACTUAL > USD 0.01; absolute threshold; USD 1 limit unchanged
Subscribers	One existing email per alert; same recipient; addresses redacted
Scope	Account-wide; Project/Environment cost tags inactive

Source: docs/EVIDENCE/01-budget/2026-10-13-budget-control-verification.json; docs/EVIDENCE/01-budget/2026-10-13-budget-settings-and-recipient-status.json; https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-managing-costs.html; https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-email-recipients.html; https://aws.amazon.com/aws-cost-management/aws-budgets/pricing/

34. Testing Strategy

LOCAL VERIFIED

Historical lifecycle checkpoint 7397f22: 756 passed, 1 skipped, zero failures; focused 62 and PostgreSQL 16 lifecycle 21. The administrator-only skip has separate prior PG16 evidence. Current multi-user verification is recorded below; neither result proves live migration 003/004/005 privileges.

Tests cover identity, threshold boundaries, immutable outcomes, rejected/quarantined rows, correction lineage, partial-write recovery, delivery claims, role/account authorization and browser workflows. Local mocks intentionally avoid AWS. Counts establish tested behavior, not availability, live service permissions or performance.

Multi-user phase: 789 regression passed / 1 administrator-only skip; 27 focused passed; 130 affected checks passed; 48 PostgreSQL 16 passed; browser passed. Focused coverage includes immutable correction lineage, failed/successful/duplicate revalidation, named onboarding and disable/reactivate, role separation, concurrent claims, wrong-owner denial, assignment replay/scope, escalated recovery and browser role/action visibility. Evidence: docs/EVIDENCE/12-e2e/multi-user-operations/.

Risk investigation: 16 new backend tests plus a real loopback browser gate; 65 affected tests; 64 on disposable PostgreSQL 16; full regression 810 passed / 1 expected administrator-only skip. Evidence: docs/EVIDENCE/12-e2e/risk-investigation/. No live AWS transport occurred.

Workbench and documentation phase: 19 focused tests passed, including three loopback browser workflows; 82 affected integration tests passed; full regression 811 passed / 1 expected administrator-only skip. The separate CSV scenario browser gate passed after its legacy review test was updated to claim before deciding. No backend, SQL or AWS changes. Evidence: docs/EVIDENCE/12-e2e/analyst-workbench/.

Local scenario phase at 48399b9: 122 lifecycle/risk/quarantine/RBAC integration and browser tests, two added outcome/role tests and one CSV A/B browser gate passed: 125 selected tests, no failures or skips. Evidence labels individual UI browser versus API checks. No fresh full regression or live AWS E2E is implied. See docs/EVIDENCE/12-e2e/scenario-verification/SCENARIO_RESULTS.md.

Source: docs/EVIDENCE/12-e2e/transaction-lifecycle/verification-summary.json; tests/test_transaction_lifecycle.py

35. CI/CD

LOCAL VERIFIED

GitHub Actions provides exact-source Linux Python 3.12 packaging/import and disposable PostgreSQL 16 gates. Dependency wheels, handlers, native PostgreSQL imports, CA loading, archive contents and checksum are tied to the release manifest.

Build/test workflows do not authorize deployment. A push and current-source CI run require their own scope. Historical green runs and artifacts are preserved; source movement and lifecycle changes require a fresh gate before promotion.

Source: infrastructure/LINUX_CI_GATE.md; .github/workflows

36. Release Verification

PENDING VERIFICATION

Freeze source commit, build inputs, package digest, manifest, Linux imports and test proof together. Verify uploaded VersionId and checksum before a change set. Never reuse an old Linux result for changed application source.

The current checkpoint is LOCAL VERIFIED. A deployable artifact still needs the exact-source Linux gate. Live schema bootstrap and migration 003 privileges, authenticated API, cloud queue/stream paths, SES, alarm delivery and recovery exercise remain gates. Approval is scoped and sequential.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; infrastructure/HARDENED_RUNTIME_RELEASE.md

37. Challenges and Failures

LOCAL VERIFIED

Live Stage 3 exposed missing DescribeEngineDefaultParameters and retained named parameter-group cleanup concerns. Stage 5 exposed provider-side tagged log creation authorization requirements that static validation could not prove; one-variable diagnostics narrowed the cause.

The read-only RDS preflight proved private verified TLS connectivity and found missing base schema, preventing an unsafe incremental migration. Quota constraints required a separately approved one-shot activation with finally restoration and revocation.

Local lifecycle verification caught retry/recovery state defects; financial storage and processing failure now remain separate. Historical outputs stay in EVIDENCE, including failures rather than only successes.

Source: infrastructure/STAGE5_PERMANENT_FIX.md; docs/EVIDENCE/05-rds/release1-live-readonly-alternative-preflight-result.md; docs/architecture/TRANSACTION_LIFECYCLE.md

38. Lessons Learned

LOCAL VERIFIED

A valid CloudFormation template does not prove live provider authorization. Simulated IAM context does not guarantee identical runtime context. Change one variable and capture both failure and recovery evidence.

Preflight must confirm the actual database baseline. Retained named resources can collide with fresh CREATE attempts. Disabled concurrency is a deployment safety control, not evidence of runtime correctness.

Financial acceptance, risk, review and notification delivery must remain independent. Never hide gaps with fabricated timestamps, UI-only labels, silent resets or indiscriminate permission broadening.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; docs/architecture/TRANSACTION_LIFECYCLE.md

39. Current Deployment State

PENDING VERIFICATION

Historical foundation evidence covers network, S3/KMS, RDS PostgreSQL and DynamoDB/SQS. Read-only live cost inventory on 12-13 October 2026 confirms those stacks and ingest are CREATE_COMPLETE, with ingest reserved concurrency 0. Existing budgets are healthy and preserved; no AWS mutation occurred. Historical deployed artifacts are distinct from current source.

The 11 October 2026 live read-only check used PostgreSQL 16.15 with TLS verify-full and found the application base schema missing. Cleanup then removed temporary runner/network stacks and billable endpoint, preserving the five existing stacks and retained evidence.

Current application source is local verified. Complete cloud integration, current-source Linux packaging and live SQL bootstrap are pending. Do not call the whole application deployed.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; docs/EVIDENCE/05-rds/release1-live-readonly-alternative-preflight-result.md; docs/EVIDENCE/05-rds/release1-temporary-preflight-cleanup-result.md

40. Future Roadmap

PLANNED

Next: exact-source Linux Python 3.12 x86_64 verification, then separately approved deployment/wiring and live AWS E2E. Local 20-scenario evidence is verified at 48399b9; AI remains planned.

After local closure: current Linux artifact gate, separately reviewed base bootstrap and 001/002/003/004/005 privilege/catalog chain, controlled live integration and recovery evidence. HA,

federation, accounting extensions, archival automation and production capacity review are later scope.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md

41. AI Investigation Assistant

PLANNED

PLANNED / NOT IMPLEMENTED. A future assistant could summarize existing authorized evidence and explain deterministic signals, with sources and human approval. It must not change ledger identity, authorize funds movement or decide confirmed fraud automatically.

No model integration, autonomous investigation, accuracy evaluation or production data transfer exists in the current release. Privacy, access boundaries, audit, quality and cost require a separate approved design.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md

42. References

LOCAL VERIFIED

Scope authority is the original user-supplied FinTrust brief. Current flow/lifecycle contracts, SQL, frozen rules, templates and evidence establish implementation status. Online sources validate engineering choices without redefining requirements. Historical narrative is subordinate to current evidence.

The document inventory classifies every authored Markdown/text/PDF. Active deployment/tooling contracts remain supporting references. Historical evidence remains unchanged; superseded clean narrative is archived where safe.

Regenerate both PDFs with python tools/build_project_docs.py after editing docs/source/project-master.json and project-progress.json. Rendering dependencies are reportlab and PyMuPDF. Update explanatory material in Master, status/history in Progress, and actual proof in EVIDENCE.

DECISION: Single owner with row lock and version | SOURCE: https://www.postgresql.org/docs/16/explicit-locking.html | WHY IT APPLIES: Concurrent staff must not overwrite ownership | HOW FINTRUST USES IT: SELECT FOR UPDATE plus expected_version and immutable history.

DECISION: Server-side role, ownership and account checks | SOURCE: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html | WHY IT APPLIES: The brief requires accountable separation of duties | HOW FINTRUST USES IT: Deny unsupported actions and validate owner/account/active role.

DECISION: Controlled local credential enrollment | SOURCE: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html | WHY IT APPLIES: Named sign-in must not expose credentials or imply production federation | HOW FINTRUST USES IT: Private prompt, unique opaque credentials, hashes only, disable checks.

DECISION: Actor and outcome audit without secrets | SOURCE: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html | WHY IT APPLIES: Investigations require attributable history | HOW FINTRUST USES IT: Reuse append-only SQL history and versioned governance receipts; no credential logging.

DECISION: list-preserving investigation drawer, compact filters and visible triage | SOURCE: https://docs.datadoghq.com/security/cloud_siem/triage_and_investigate/investigate_security_signals/ | WHY IT APPLIES: Analysts need evidence and next steps without losing queue context | HOW FINTRUST USES IT: Adapt interaction principles to transaction risk, with original branding and backend-enforced ownership; no copied icons, wording or pixel layout.

DECISION: Reuse scoped aggregate and queue reads | SOURCE: backend/fintrust/frontend_workflows.py; visual_insights.py; workflow_api.py | WHY IT APPLIES: The brief requires evidence-backed triage without fabricated totals | HOW FINTRUST USES IT: Independent loading/error states, explicit count bounds and unconfigured SLA handling; no new risk or database model.

DECISION: Preserve existing monthly and zero-spend budget controls | SOURCE: AWS Budgets managing-costs and email-recipient guidance | WHY IT APPLIES: Avoid duplicate alerts while proving actual threshold/scope configuration | HOW FINTRUST USES IT: Account-wide read-only verification, masked configured subscriber evidence, periodic-notification limitations and separate live deployment approval.

Source: docs/architecture/FINTRUST_FLOW_AUDIT.md; docs/architecture/TRANSACTION_LIFECYCLE.md; docs/source/documentation-inventory.json; https://www.postgresql.org/docs/16/explicit-locking.html; https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html; https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html; https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html; https://docs.datadoghq.com/security/cloud_siem/triage_and_investigate/investigate_security_signals/; sql/migrations/005_investigation_notes.sql; backend/fintrust/investigation.py; docs/EVIDENCE/12-e2e/risk-investigation/verification-summary.json; docs/EVIDENCE/12-e2e/analyst-workbench/verification-summary.json; https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-managing-costs.html; https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-email-recipients.html; https://aws.amazon.com/aws-cost-management/aws-budgets/pricing/; docs/EVIDENCE/01-budget/2026-10-13-budget-control-verification.json